

Guía didáctica – Módulo 2

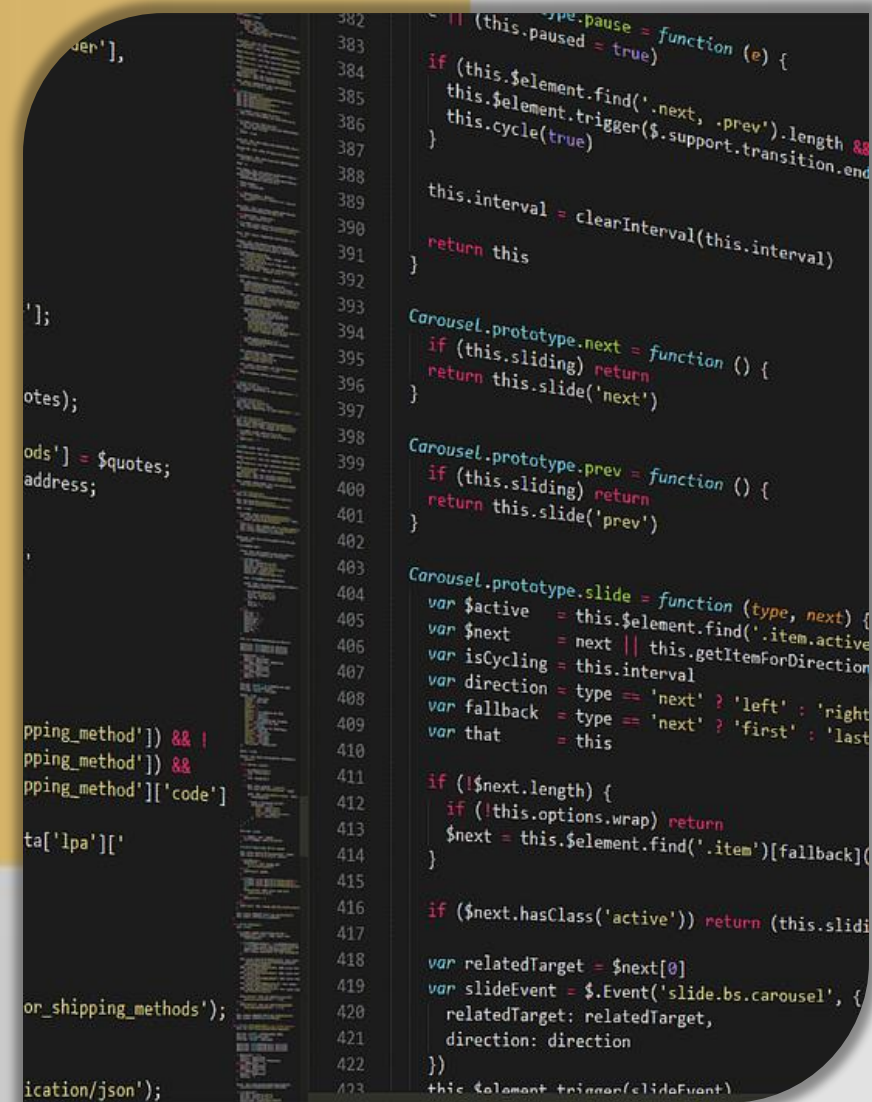


Imagen: Pixabay



Guía didáctica

**Strings, objetos,
estructuras de
control y ciclos**

JS



Competencia específica

Se espera que, con los temas abordados en la guía didáctica del módulo 2: Strings, objetos, estructuras de control y ciclos, el estudiante logre la siguiente competencia específica:

Comprender las diferentes estructuras condicionales y de iteración, además del uso de los objetos para estructurar y almacenar información.



Contenidos temáticos

Los siguientes son los contenidos temáticos para desarrollar en la guía didáctica del módulo 2: Strings, objetos, estructuras de control y ciclos:

1

Strings

2

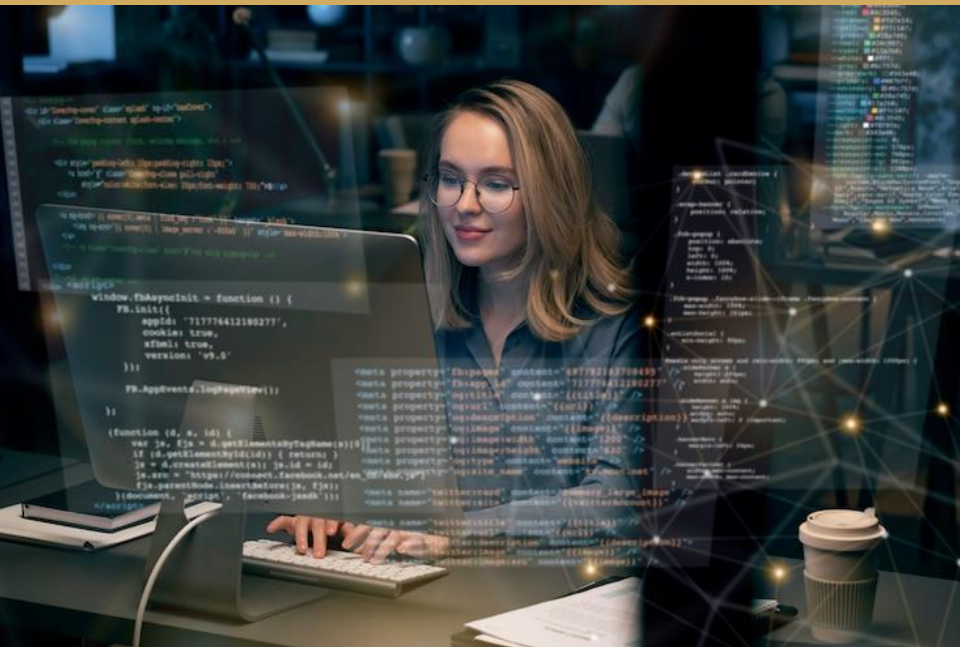
Objetos

3

Estructuras de control

4

Ciclos



Activación de saberes previos

Antes de iniciar con el estudio de la guía didáctica, reflexiona sobre las siguientes preguntas:

- ¿Conoces otro lenguaje de programación?
- ¿Cómo te han parecido los conceptos básicos?

No inicies el segundo módulo sin tener claro lo visto en la guía didáctica del módulo 1.



Imagen: Freepik

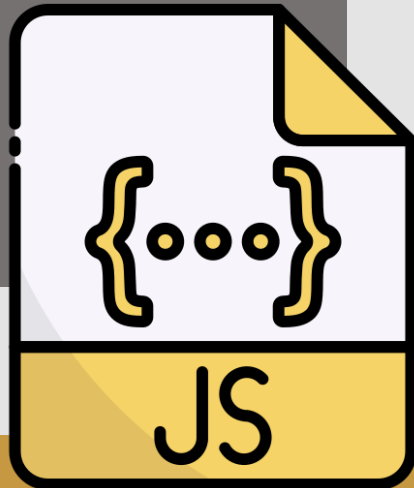
Tema 1:

Strings

«El avance de la tecnología se basa en hacerla encajar para que ni siquiera te des cuenta, por lo que es parte de la vida cotidiana». Bill Gates

Strings

En JavaScript, después de conocer los tipos de datos, y puntualmente los **strings** o **cadenas de texto**, es importante conocer los métodos que ofrece esta interfaz para operar las cadena de texto.



Este tipo de dato o **interfaz** ofrece diversos **métodos** para operar las cadenas de texto, desde obtener la longitud de una cadena, hasta dividir la misma. A continuación, se presentan los métodos y su implementación.

(.length)

Retorna la longitud de la cadena, es decir, el número de caracteres que conforma el dato.



```
1  const nombre = "Diego Palacio"
2
3  console.log(nombre.length) // 13
```

charAt

Retorna el carácter en la posición especificada.



```
1  const nombre = "Diego Palacio"
2
3  console.log(nombre.charAt(3)) // g
```


concat

Permite unir o concatenar cadenas o unir diferentes strings.

```
1  const nombre = "Diego"
2  const apellido = "Palacio"
3
4  const nombreCompleto = nombre.concat(" ", apellido)
5
6  console.log(nombreCompleto)
7
8  // Diego Palacio
```

includes

Retorna un booleano en caso de existir una coincidencia en la cadena.

```
1  const nombre = "Diego"
2
3  console.log(nombre.includes("ego")) // true
4  console.log(nombre.includes("Pal")) // false
```

substrings

Retorna una subcadena de la cadena original, desde la posición de inicio hasta la posición de fin especificadas.

```
1 const nombre = "Diego"
2
3 console.log(nombre.substring(0, 2)) // Di
4 console.log(nombre.substring(1, 3)) // ie
5 console.log(nombre.substring(2, 4)) // eg
6 console.log(nombre.substring(3, 5)) // go
7 console.log(nombre.substring(4, 6)) // o
8 console.log(nombre.substring(2)) // ego
9
```

slice

Separa la cadena en un arreglo de subcadenas, utilizando el separador especificado.

```
1 const nombre = "Diego"
2
3 console.log(nombre.slice(0, 2)) // Di
4 console.log(nombre.slice(2, 5)) // ego
5 console.log(nombre.slice(0, 5)) // Diego
6 console.log(nombre.slice(0, 1)) // D
7 console.log(nombre.slice(-1)) // o
8 console.log(nombre.slice(-2, -1)) // g
```

toUpperCase

Retorna la cadena en mayúsculas.



```
1  const nombre = "Diego"
2
3  console.log(nombre.toUpperCase())
4
5  // DIEGO
```

toLowerCase

Retorna la cadena en minúsculas.



```
1  const nombre = "Diego"
2
3  console.log(nombre.toLowerCase())
4
5  // diego
```

replace

Reemplaza la primera ocurrencia de la subcadena especificada con la cadena de reemplazo especificada.



```
1 const nombre = "Diego"
2
3 console.log(nombre.replace("Diego", "Juan")) // Juan
```



```
1 const nombre = "Diego"
2
3 console.log(nombre.replace("ego", "plo")) // Diplo
```

replaceAll

Reemplaza todas las ocurrencias de la subcadena especificada con la cadena de reemplazo especificada.



```
1 const nombre = "Diego Palacio"
2
3 console.log(nombre.replaceAll("o", "a"))
4
5 // Diaga Palacia
```


split

Separa la cadena en un arreglo de subcadenas, utilizando el separador especificado.

```
1  const nombre = "Diego Palacio"
2
3  console.log(nombre.split(' '))
4
5  // [ 'D', 'i', 'e', 'g', 'o', ' ', 'P', 'a', 'l', 'a', 'c', 'i', 'o' ]
6
```

repeat

Devuelve una nueva cadena que contiene el número especificado de copias de la cadena original.

```
1  const nombre = "Diego"
2
3  console.log(nombre.repeat(3))
4
5  // DiegoDiegoDiego
6
```

endsWith

Devuelve un valor booleano que indica si la cadena termina con los caracteres de la subcadena especificada.



```
1  const nombre = "Diego PAlacio"
2
3  console.log(nombre.endsWith("o")) // true
4  console.log(nombre.endsWith("z")) // false
5  console.log(nombre.endsWith("Palacio")) // true
```

startsWith

Devuelve un valor booleano que indica si la cadena empieza con los caracteres de la subcadena especificada.



```
1  const nombre = "Diego PAlacio"
2
3  console.log(nombre.startsWith("Diego")) // true
4  console.log(nombre.startsWith("diego")) // false
5  console.log(nombre.startsWith("D")) // true
6  console.log(nombre.startsWith("d")) // false
```



Practica lo aprendido

Pon en práctica lo visto acerca de variables resolviendo los siguientes ejercicios:

Ejercicio 1

Crea una variable con el valor de «Diego Alejandro Palacio Valencia». ¿Qué función se podría utilizar para determinar la longitud/tamaño?

Ejercicio 2

Crea una variable con el valor de «Pinocho» y deletrea accediendo a cada una de las letras de la palabra «Pinocho».

Ejercicio 3

Crea una variable para almacenar un nombre completo; existen 2 variables para los nombres y 2 variables para los apellidos. Utiliza la función concat.

Ejercicio 4

Crea una variable con el valor de «Politécnico de Colombia» y busca alguna coincidencia con la cadena «co».

Practica lo aprendido



Ejercicio 5

Almacena en una variable la siguiente cadena
«IMPLEMENTAR –
230330-230501-C380»,
separar la cadena
donde obtenga
únicamente
«IMPLEMENTAR».

Ejercicio 6

Convierte la cadena de
«estados unidos» a
«ESTADOS UNIDOS».

Ejercicio 7

Convierte la cadena de
«ESTADOS UNIDOS»
a «estados unidos».

Ejercicio 8

A partir de una variable
con los siguientes
valores «😍❤️😍❤️»,
reemplaza los
corazones por «😭».

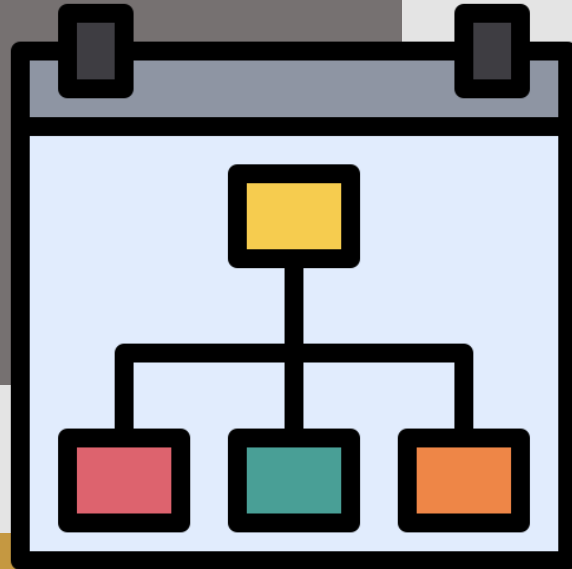
Tema 2:

Objetos

«Las pruebas conducen al fracaso y el fracaso conduce a la comprensión». Burt Rutan

Objetos

Los objetos en JavaScript almacenan un **conjunto** de **propiedades** y **valores** relacionados o también conocidos como **clave - valor**. Los objetos se **declaran** utilizando la sintaxis de **llaves**.



Declaración

Propiedades

Métodos

Operando

Declaración

Para declarar un objeto, se usan las **llaves** `{}` y dentro, las **propiedades** y **métodos** separados por comas. Cada **propiedad** o **método** se define con una **clave** y un **valor** separados por dos puntos:



```
1  const persona = {  
2      nombre: "Diego",  
3      edad: 25,  
4      empleado: true  
5  }
```

Las propiedades y métodos de un objeto pueden ser de cualquier tipo de JavaScript, incluso otros objetos o arrays.



```
1  const persona = {  
2      nombre: "Diego",  
3      edad: 25,  
4      empleado: true,  
5      familia: ["Ana", "Pedro", "Juan"],  
6      direccion: {  
7          calle: "Calle 123",  
8          numero: 123,  
9          ciudad: "Medelín"  
10     }  
11 }
```

Propiedades

En JavaScript, las propiedades de un objeto son los atributos o características que definen a ese objeto. Cada propiedad se representa como un par clave-valor dentro del objeto, donde:

Clave: es el nombre de la propiedad y puede ser un string o un símbolo.

Valor: es el contenido de la propiedad, que puede ser cualquier tipo de dato (números, strings, arrays, otros objetos, funciones, etc.).

```
1  const casa = {  
2      pisos: 2,  
3      habitaciones: 3,  
4      baños: 2,  
5      direccion: {  
6          calle: 'Calle 123',  
7          ciudad: 'Bogotá',  
8          pais: 'Colombia'  
9      }  
10 }
```

En este ejemplo, el objeto cuenta con 4 propiedades (pisos, habitaciones, baños y dirección) con sus respectivos valores, siendo dirección otro objeto con 3 propiedades.

Métodos

Hasta ahora las **propiedades** de los objetos han sido de los tipos más comunes, strings, numbers, objects, etc., pero otra característica de los objetos es que pueden contener **métodos**, agregando así funcionalidad a los objetos.

Un **método** es como una función normal, pero está asociada a un objeto específico. Esto permite encapsular tanto los datos como las funciones que operan sobre esos datos dentro del mismo objeto.

```
1  const personaje = {
2    nombre: "Juak21",
3    puntosVida: 100,
4    ataque: 10,
5    defensa: 5,
6    atacar: function () {
7      console.log("Atacando")
8    },
9    defender: function () {
10     console.log("Defendiendo")
11   }
12 }
```

```
1  const rectangulo = {
2    base: 10,
3    altura: 20,
4    area: function() {
5      console.log(this.base * this.altura)
6    },
7    perimetro: function() {
8      console.log(2 * this.base + 2 * this.altura)
9    }
10 }
```

Operando objetos

Con los conceptos básicos sobre la declaración y **funcionamiento** de los objetos, se abre un abanico de posibilidades en cuanto a cómo y qué **operaciones** se pueden realizar con los mismos en casos reales, desde el acceso a las **propiedades** (para imprimir, modificar o eliminar) hasta la protección y unión de los objetos.

A continuación, se detallan las posibles **operaciones** que se pueden realizar con los objetos.

Creación

Acceso

Modificar

Eliminar

Destructuring

Objetos en objetos

Proteger

Unir

Creación de objetos

Para crear un objeto en **JavaScript**, podemos usar la sintaxis de llaves ({}) y definir sus **propiedades** y **valores** dentro de las llaves.

Por ejemplo:

```
1  const personaje = {  
2      nombre: 'Goku',  
3      raza: 'Saiyajin',  
4      edad: 35,  
5      poder: 9000  
6  }
```

Acceder al objeto

Para acceder a las **propiedades** podemos usar la notación de punto (objeto.**propiedad**) o la notación de corchetes (objeto["propiedad"]).

Por ejemplo:

```
1  const personaje = {  
2      nombre: 'Goku',  
3      raza: 'Saiyajin',  
4      edad: 35,  
5      poder: 9000  
6  }  
7  
8  console.log(personaje) // Todo el objeto  
9  console.log(personaje.nombre) // Goku  
10 console.log(personaje.raza) // Saiyajin  
11 console.log(personaje.edad) // 35  
12 console.log(personaje.poder) // 9000
```

Agregar propiedades

Para agregar propiedades a un objeto, podemos usar la **notación** de **punto** (objeto.**nuevaPropiedad** = valor).

Por ejemplo:

```
1  const personaje = {
2      nombre: 'Goku',
3      raza: 'Saiyajin',
4      edad: 35,
5      poder: 9000
6  }
7
8  personaje.hermano = 'Raditz'
9
10 console.log(personaje)
```

Modificar propiedades

Para modificar propiedades de un objeto en **JavaScript**, usamos la notación de punto (objeto.propiedad = nuevoValor) o la **notación** de **corchetes** (objeto["propiedad"] = nuevoValor).

Por ejemplo:

```
1  const personaje = {
2      nombre: 'Goku',
3      raza: 'Saiyajin',
4      edad: 35,
5      poder: 9000
6  }
7
8  personaje.edad = 55
9
10 console.log(personaje)
```

Destructuring

Para realizar **destructuring** de un objeto, utilizamos la sintaxis de llaves para extraer **propiedades** específicas en variables individuales. Por ejemplo, `const { nombre, edad } = persona` crea variables `nombre` y `edad` con los valores correspondientes del objeto `persona`.

El **destructuring** es una forma de extraer valores de un objeto y asignarlos a variables de manera más concisa. Esto simplifica el acceso a los datos y reduce la cantidad de código cuando necesitamos extraer varias **propiedades** de un objeto.

```
1  const personaje = {
2      nombre: 'Goku',
3      raza: 'Saiyajin',
4      edad: 35,
5      poder: 9000
6  }
7
8  const nombre = personaje.nombre
9  const raza = personaje.raza
10
11 const { edad, poder } = personaje
```

Eliminar propiedades

Para eliminar propiedades de un objeto, utilizamos la palabra clave **delete** seguida de la propiedad para eliminar.

Por ejemplo, `delete persona.edad` quitará la **propiedad** edad del objeto persona.

```
1  const personaje = {  
2      nombre: 'Goku',  
3      raza: 'Saiyajin',  
4      edad: 35,  
5      poder: 9000  
6  }  
7  
8  delete personaje.edad  
9  
10 console.log(personaje)
```


Objetos en objetos

En **JavaScript**, podemos crear objetos dentro de otros objetos **asignando** un objeto como valor de una **propiedad**.

Por ejemplo:

```
1  const personaje = {
2    nombre: 'Goku',
3    raza: 'Saiyajin',
4    edad: 35,
5    poder: 9000,
6    informacion: {
7      planeta: 'Tierra',
8      serie: 'Dragon Ball Z',
9      coordenadas: {
10        latitud: 12.3456,
11        longitud: 45.6789
12      }
13    }
14  }
15
16  console.log(personaje.informacion.planeta); // Tierra
```

Destructuring

```
1  const personaje = {
2    nombre: 'Goku',
3    raza: 'Saiyajin',
4    edad: 35,
5    poder: 9000,
6    informacion: {
7      planeta: 'Tierra',
8      serie: 'Dragon Ball Z',
9      coordenadas: {
10        latitud: 12.3456,
11        longitud: 45.6789
12      }
13    }
14  }
15
16  const { nombre, informacion: { planeta, serie } } = personaje
17
18  console.log(nombre)
19  console.log(planeta)
20  console.log(serie)
21
```

Proteger objetos

Para **proteger** objetos y **evitar** modificaciones no deseadas, podemos usar el método `Object.freeze(objeto)`, que impide que se añadan, eliminen o modifiquen **propiedades** del objeto, haciendo que se vuelva inmutable.

Por ejemplo:

```
1  "use strict"
2
3  const personaje = {
4    nombre: 'Goku',
5    raza: 'Saiyajin',
6    edad: 35,
7    poder: 9000
8  }
9
10 Object.freeze(personaje)
11
12 personaje.nombre = 'Vegeta' // Error
13
14 personaje.imagen = 'goku.jpg' // Error
15
16 delete personaje.edad // Error
```

Sellar objetos

Para **sellar** objetos, utilizamos el método `Object.seal(objeto)`, que permite mantener las propiedades existentes, pero impide que se añadan nuevas **propiedades** o que se **eliminen** las existentes.

Por ejemplo:

```
1  "use strict"
2
3  const personaje = {
4    nombre: 'Goku',
5    raza: 'Saiyajin',
6    edad: 35,
7    poder: 9000
8  }
9
10 Object.seal(personaje)
11
12 personaje.nombre = 'Vegeta' // nombre: Vegeta
13
14 personaje.imagen = 'goku.jpg' // Error
15
16 delete personaje.edad // Error
17
```

Unir objetos

Para unir objetos, utilizamos el método `Object.assign(destino, ...orígenes)`, que copia las propiedades de uno o más **objetos** de origen en un objeto destino.

Por ejemplo:

```
1  const personaje = {
2    nombre: 'Goku',
3    raza: 'Saiyajin',
4    edad: 35,
5    poder: 9000
6  }
7
8  const informacion = {
9    planeta: 'Tierra',
10   saga: 'Cell',
11   personajes: ['Goku', 'Vegeta', 'Gohan']
12 }
13
14 const personajeFinal = Object.assign(personaje, informacion)
15
16 console.log(personajeFinal)
```

¿This?

El método **this** dentro de un objeto se refiere al propio objeto y se utiliza para acceder a sus **propiedades** y **métodos**.

Por ejemplo:

```
1  const personaje = {
2    nombre: 'Goku',
3    raza: 'Saiyajin',
4    edad: 35,
5    poder: 9000,
6    informacion: function() {
7      console.log(`Personaje ${nombre}`)
8    }
9  }
10
11 personaje.informacion() // Error: nombre no existe
```

```
1  const nombre = "Batman"
2
3  const personaje = {
4    nombre: 'Goku',
5    raza: 'Saiyajin',
6    edad: 35,
7    poder: 9000,
8    informacion: function() {
9      console.log(`Personaje ${nombre}`)
10     }
11  }
12
13 personaje.informacion() // Batman ¿?
```

```
1  const nombre = "Batman"
2
3  const personaje = {
4    nombre: 'Goku',
5    raza: 'Saiyajin',
6    edad: 35,
7    poder: 9000,
8    informacion: function() {
9      console.log(`Personaje ${this.nombre}`)
10     }
11  }
12
13 personaje.informacion() // Personaje Goku
```

Keys

Para **acceder** a las propiedades de un objeto utilizando `Object.keys(objeto)`, obtenemos un array que contiene los nombres de las propiedades **enumerables** del objeto, lo que nos permite iterar sobre ellas o verificar qué propiedades existen.

Por ejemplo:

```
1  const personaje = {  
2      nombre: 'Goku',  
3      raza: 'Saiyajin',  
4      edad: 35,  
5      poder: 9000,  
6  }  
7  
8  console.log(Object.keys(personaje))  
9  
10 // ["nombre", "raza", "edad", "poder"]  
11
```

Values

Para obtener los **valores** de las **propiedades** de un objeto, podemos utilizar `Object.values(objeto)`, que devuelve un array que contiene todos los valores de las **propiedades** numerables del objeto.

Por ejemplo:



```
1  const personaje = {
2      nombre: 'Goku',
3      raza: 'Saiyajin',
4      edad: 35,
5      poder: 9000,
6  }
7
8  console.log(Object.values(personaje))
9
10 // ["nombre", "raza", "edad", "poder"]
```

Entries

Para obtener tanto las **claves** como los valores de un objeto, utilizamos `Object.entries(objeto)`, que devuelve un array de pares clave-valor, donde cada par es un **array** que contiene el nombre de la propiedad y su correspondiente **valor**.

Por ejemplo:

```
1  const personaje = {
2      nombre: 'Goku',
3      raza: 'Saiyajin',
4      edad: 35,
5      poder: 9000,
6  }
7
8  console.log(Object.entries(personaje))
9
10 // [
11 //     [ 'nombre', 'Goku' ],
12 //     [ 'raza', 'Saiyajin' ],
13 //     [ 'edad', 35 ],
14 //     [ 'poder', 9000 ]
15 // ]
```

Ejercicio 1

Practica lo aprendido

Para practicar lo visto sobre objetos, realiza los siguientes ejercicios:



Imagen: Freepik

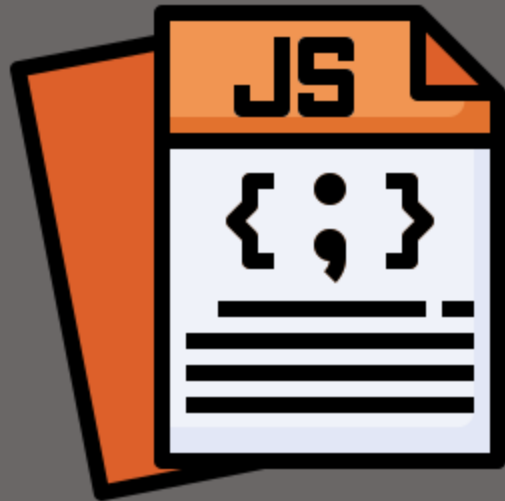
Crea un objeto para almacenar la información de contacto de una persona. Se requiere que este objeto tenga una propiedad para el nombre completo, el número de teléfono y el correo electrónico.

Muestra el objeto completo y cada una de sus propiedades.

Ejercicio 2

Crear un objeto usuario con datos y preferencias. Este objeto debe tener una propiedad para almacenar el nombre, la edad y el tema (claro u oscuro) de preferencia de una interfaz. Usa destructuring para extraer las propiedades nombre y tema, y muestra un mensaje que diga: «El usuario <nombre> prefiere el tema <tema>».

Practica lo aprendido



Ejercicio 3

Crea un objeto `cuentaBancaria` que permita a los usuarios consultar el saldo, hacer depósitos y retiros y ver el historial de transacciones.

Ejercicio 4

Crea un objeto `producto` que almacene detalles de un producto específico, con propiedades como el nombre, precio y cantidad en *stock*, además de un método para aplicar un descuento.

Tema 3:

Estructuras de control

«Caminar sobre el agua y desarrollar *software* con base en una especificación es sencillo, si ambos están congelados». Edward Berard



Estructuras de control

Las estructuras de control en JavaScript son herramientas que permiten controlar el **flujo** de ejecución de un programa.

Estas **estructuras** facilitan la toma de decisiones y la ejecución de acciones repetidas en función de las condiciones establecidas.

Condicional if

Condicional if/else

Condicional if/else/if

Operador ternario

Condicionales anidadas

Switch

Diccionario de datos

Condicional if

Esta estructura permite tomar **decisiones** en función de una condición.

Se utiliza para ejecutar un bloque de **código** si una **condición** es verdadera.

```
if ( condición ) {  
    bloque de ejecución  
}
```

```
1  const persona = {  
2      nombre: 'Juan',  
3      apellido: 'Perez',  
4      edad: 28,  
5      direccion: {  
6          ciudad: 'CDMX',  
7          zip: 12345,  
8      }  
9  }  
10  
11  if (persona.edad ≥ 18){ // true  
12      console.log('Eres mayor de edad')  
13      // Eres mayor de edad  
14  }  
15  
16  if (persona.nombre === 'Juan'){ // true  
17      console.log('Hola Juan')  
18      // Hola Juan  
19  }  
20  
21  if (persona.apellido === 'Palacio'){ // false  
22      console.log(`Hola ${persona.nombre} Palacio`)  
23      // No se ejecuta  
24  }  
25  
26  if (persona.edad ≥ 18 && persona.direccion.ciudad === 'CDMX'){ // true  
27      console.log(`Hola ${persona.nombre} ${persona.apellido}  
28          eres mayor de edad y vives en la CDMX`)  
29      // Hola Juan Perez eres mayor de edad y vives en la CDMX  
30  }
```

Condicional if/else

Esta estructura permite **ejecutar** un **bloque** de **código** si una condición es **verdadera** y otro bloque de **código** si la condición es falsa por defecto.

```
if ( condición ) {  
    bloque por cumplimiento  
} else {  
    bloque por defecto  
}
```

```
1  const institucion = {  
2      nombre: 'Politécnico de Colombia',  
3      jornadas: ['Mañana', 'Tarde', 'Noche'],  
4      sede: {  
5          nombre: 'Medellín',  
6          direccion: 'Calle 10 # 10 - 10',  
7          telefono: '1234567'  
8      }  
9  }  
10  
11  if (institucion.sede.nombre === 'Bogotá'){ // false  
12      console.log('La sede es Bogotá')  
13  } else {  
14      console.log('La sede no es Bogotá')  
15      // La sede no es Bogotá  
16  }  
17  
18  if (institucion.jornadas[0] === 'Mañana'){ // true  
19      console.log('La jornada disponible es Mañana');  
20      // La jornada disponible es Mañana  
21  } else {  
22      console.log('La jornada disponible no es Mañana')  
23  }
```

Condicional if/else/if

Esta estructura, al igual que **if/else**, permite ejecutar un bloque de código si una condición es **verdadera** y otro bloque en caso de que la condición sea **falsa**, siempre y cuando se compare otra condición.

```
if ( primera condición ) {  
    bloque por cumplimiento  
} else if( segunda condición ) {  
    bloque por cumplimiento  
} else {  
    bloque por defecto  
}
```

```
if ( primera condición ) {  
    bloque por cumplimiento  
} else if( segunda condición ) {  
    bloque por cumplimiento  
} else if( tercera condición ) {  
    bloque por cumplimiento  
} else {  
    bloque por defecto  
}
```

Condicional if/else/if

```
1  const institucion = {
2    nombre: 'Politécnico de Colombia',
3    jornadas: ['Mañana', 'Tarde', 'Noche'],
4    sede: {
5      nombre: 'Cartagena',
6      direccion: 'Calle 10 # 10 - 10',
7      telefono: '1234567'
8    }
9  }
10
11  if (institucion.sede.nombre === 'Medellín') // false
12    console.log('Bienvenido a la sede de Medellín')
13  else if (institucion.sede.nombre === 'Bogotá') // false
14    console.log('Bienvenido a la sede de Bogotá')
15  else if (institucion.sede.nombre === 'Cali') // false
16    console.log('Bienvenido a la sede de Cali')
17  else // true
18    console.log('Bienvenido a la sede de '
19      + institucion.sede.nombre)
20
21  // Bienvenido a la sede de Cartagena
```

Operador ternario

El operador ternario es una **alternativa** de condicional **if/else** de una forma mucho más corta y, en muchos casos, más **legible**.

`const result = condition ? true : false`

```
if ( condición ) {  
    bloque por cumplimiento  
} else {  
    bloque por defecto  
}
```

```
1  const pokemon = {  
2      name: 'Pikachu',  
3      type: 'Electric',  
4      number: 25,  
5      hp: 35,  
6      attack: 55,  
7      moves: ['Thunder Shock', 'Quick Attack'],  
8  }  
9  
10 if (pokemon.hp > 50 && pokemon.hp ≤ 100) {  
11     console.log('Pikachu is in good health')  
12 } else {  
13     console.log('Pikachu is in poor health')  
14 }  
15  
16 const statePokemon =  
17     pokemon.hp > 50 && pokemon.hp ≤ 100  
18     ? 'Pikachu is in good health'  
19     : 'Pikachu is in poor health'  
20  
21 console.log(statePokemon)  
22  
23 // Pikachu is in poor health
```


Condicionales anidadas

Esta estructura permite **crear** una **anidación** de condicionales **if/else/if** de forma **consecutiva**, contando así con **diferentes** comprobaciones encadenadas a un **resultado** verdadero.

Sin embargo, **anidar** de esta forma varios **if** suele ser muy **poco legible** y produce un código algo feo. En algunos casos se podría utilizar otra estructura de control llamada **switch**, que puede ser útil en ciertos casos.

```
1  const notaObtenida = 4
2
3  if (notaObtenida === 0) {
4      console.log('No aprobaste')
5  } else if (notaObtenida === 1) {
6      console.log('Insuficiente')
7  } else if (notaObtenida === 2) {
8      console.log('Suficiente')
9  } else if (notaObtenida === 3) {
10     console.log('Bien')
11 } else if (notaObtenida === 4) {
12     console.log('Notable')
13 } else if (notaObtenida === 5) {
14     console.log('Sobresaliente')
15 } else {
16     console.log('Nota no válida')
17 }
```

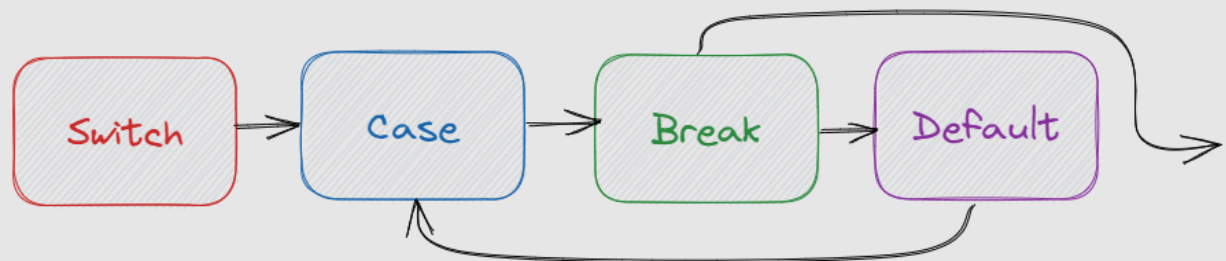
Switch

La estructura de control **switch** permite definir casos **específicos**, en el caso de que la variable expuesta como **condición** sea igual a los valores que se **especifican**, el caso será verdadero.

```
switch ( variable ){  
    case: caso  
        bloque de código  
        break // romper ejecución  
    case: caso  
        bloque de código  
        break // romper ejecución  
    default  
        bloque de ejecución  
}
```

La expresión es **evaluada** y compara con los **diferentes valores** especificados en los casos (**case**). Si la expresión es igual a uno de los valores, se **ejecuta** el **bloque** de código correspondiente. Si la expresión no es igual a ninguno de los valores especificados en los **casos**, se ejecuta el bloque de código del **default**.

El **break** es necesario después de cada bloque de código para **evitar** que se ejecuten accidentalmente otros bloques de código en el **switch**.



Switch

```
1  const notaObtenida = 4
2
3  switch (notaObtenida) {
4      case 0:
5          console.log('No aprobaste')
6          break
7      case 1:
8          console.log('Insuficiente')
9          break
10     case 2:
11         console.log('Suficiente')
12         break
13     case 3:
14         console.log('Bien')
15         break
16     case 4:
17         console.log('Notable')
18         break
19     case 5:
20         console.log('Sobresaliente')
21         break
22     default:
23         console.log('Nota no válida')
24 }
25
```

```
1  const notaObtenida = 2
2
3  switch (notaObtenida) {
4      case 0:
5      case 1:
6      case 2:
7          console.log('Muy insuficiente')
8          break
9      case 3:
10     case 4:
11     case 5:
12         console.log('Sobresaliente')
13         break
14     default:
15         console.log('Nota no válida')
16 }
```

Diccionarios de datos

En JavaScript, los **diccionarios** de **datos** se implementan utilizando **objetos**.

Un **objeto** en JavaScript es una **colección** de pares **clave-valor**, donde cada **clave** es una cadena (o un símbolo a partir de ES6) y cada **valor** puede ser de cualquier tipo de dato en JavaScript, incluyendo otros objetos.

Para **acceder** a los valores de un **objeto**, podemos utilizar la **notación** de **corchetes** o la notación de **punto**.

Esto permite en muchos casos **optimizar** las **estructuras condicionales** a bloques mucho más **pequeños** y **legibles** de código.



```
1  const PAISES = {  
2      colombia: "CO",  
3      venezuela: "VE",  
4      ecuador: "EC",  
5      peru: "PE",  
6      chile: "CL",  
7      argentina: "AR",  
8  }
```

Diccionarios de datos

```
1 const pais = "Italia"
2
3 if (pais === "Italia") {
4     console.log("IT")
5 } else if (pais === "España") {
6     console.log("ES")
7 } else if (pais === "Francia") {
8     console.log("FR")
9 } else if (pais === "Alemania") {
10     console.log("DE")
11 }
```

```
1 const pais = "Italia"
2
3 switch (pais) {
4     case "Italia":
5         console.log("IT")
6         break;
7     case "España":
8         console.log("ES")
9         break;
10    case "Francia":
11        console.log("FR")
12        break;
13    case "Alemania":
14        console.log("DE")
15        break;
16    default:
17        console.log("No se encontro el pais")
18        break;
19 }
```

```
1 const pais = "Italia"
2
3 const PAISES = {
4     Espana: "ES",
5     Italia: "IT",
6     Francia: "FR",
7     Alemania: "DE"
8 }
9
10 console.log(PAISES[pais]) // IT
```

Practica lo aprendido

Pon en práctica lo visto acerca de estructuras de control resolviendo los siguientes ejercicios:

Ejercicio 1

Una tienda ofrece descuentos dependiendo de la cantidad de productos que se compran. Si se compran más de 10 productos, se tiene un 20% de descuento; si se compran entre 5 y 10, se obtiene un 10%, y si se compran menos de 5, no hay descuento.

Crea las variables necesarias para satisfacer el ejercicio.

Ejercicio 2

Se necesita saber si un estudiante aprobó o no. Un estudiante aprueba con una nota igual o superior al 60%. Se tienen las siguientes notas:

- Parcial del 25%
- Quiz del 20%
- Talleres del 30%
- Parcial del 25%

Calcula un promedio para un estudiante X y asigna notas al azar para cada una de las actividades.

Ejercicio 3

Una empresa tiene almacenada la información de sus empleados en diccionarios de datos. La empresa otorgará unos bonos a los empleados que tuvieron más de 60 ventas.

Determina si un empleado tiene o no bono.

Ejercicio 4

A partir del nombre de una fruta, indica la categoría de color a la que pertenece. Por ejemplo: las manzanas y las fresas son rojas.

Ejercicio 5

Usando el operador ternario, determina si un número es par o impar.

Ejercicio 6

Un restaurante tiene un menú en el que cada plato tiene su precio guardado en un objeto. Dependiendo del plato que un cliente elija, el programa debe mostrar el precio de este. Si el plato no está en el menú, se debe mostrar un mensaje indicando que no está disponible.



Ejercicio 7

En una tienda de electrodomésticos venden diferentes tipos de productos, como teléfonos, *laptops* y consolas. Se necesita saber cuánto cuesta el envío según el tipo de producto: los teléfonos tienen un costo de envío de \$5, las *laptops* de \$10, y las consolas de \$15.

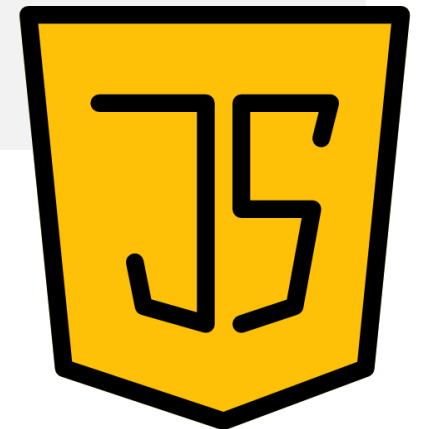
Utiliza el operador switch para ayudar a calcular el costo de envío.

Ejercicio 8

En una carrera infantil, los niños se clasifican por grupos según su edad: de 0 a 5 años son «bebés», de 6 a 10 son «niños», y de 11 a 15 son «preadolescentes». Determinar en qué grupo va cada participante.

Utiliza estructuras if-else para hacer la clasificación.

Imagen: Flaticon



Ejercicio 9

Una vendedora tiene que decidir si ir a visitar clientes en automóvil o en bicicleta. Si la distancia es mayor a 10 km, elige el automóvil; de lo contrario, prefiere la bicicleta.

Utiliza el operador la tomar la decisión basada en la distancia que ingrese.

Ejercicio 10

Asigna una letra según el puntaje de calificación de un grupo de estudiantes. Si el puntaje es mayor o igual a 90, la calificación es «A», si es mayor o igual a 80 es «B», si es mayor o igual a 70 es «C», y si es menor a 70 es «D».

Implementa esta lógica usando if-else.



Tema 4:

Ciclos

«Medir el progreso del desarrollo de *software* por líneas de código es como medir el progreso de la construcción de un avión por su peso». Bill Gates

Estructuras cíclicas

Los **ciclos** o **bucles** en JavaScript se utilizan para **repetir** un **bloque** de **código** varias veces. Hay varios tipos de **ciclos** en JavaScript, cada uno con su propia **sintaxis** y uso.

Antes de comenzar a ver qué **tipos** de **bucles** existen en JavaScript, es necesario conocer algunos conceptos básicos de los bucles:

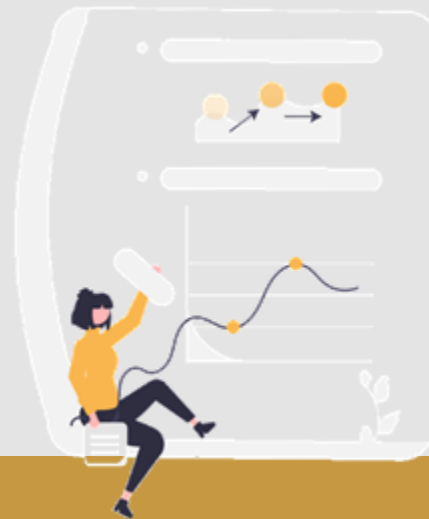
Condición

Iteración

Contador

Incremento

Bucle infinito



Conceptos básicos de los bucles

Condición

Al igual que en los if, en los **bucles** se va a **evaluar** una **condición** para saber si se debe **repetir** el bucle o **finalizarlo**.

Generalmente, si la condición es **verdadera**, se **repite**. Si es **falsa**, se **finaliza**.

Iteración

Cada **repetición** de un **bucle** se denomina **iteración**.

Por ejemplo, si un bucle **repite** una acción 10 veces, se dice que tiene 10 **iteraciones**.

Conceptos básicos de los bucles

Contador

Muchas veces, los **bucles** tienen una **variable** que se denomina **contador**, porque cuenta el número de **repeticiones** que ha hecho para **finalizar** desde que llegue a un número concreto.

Dicha variable hay que inicializarla (**crearla** y darle un **valor**) antes de comenzar el **bucle**.

Incremento

Cada vez que se **termine** una iteración se suele realizar el **incremento** (o **decremento**) de una variable, generalmente la denominada variable **contador**.



Imagen: Freepik

Conceptos básicos de los bucles

Bucle infinito

Es lo que ocurre si en un bucle se nos olvida **incrementar** la variable **contador** o escribimos una **condición** que **nunca** se puede dar. El **bucle** se queda **eternamente repitiéndose** y el programa se queda «**colgado**».

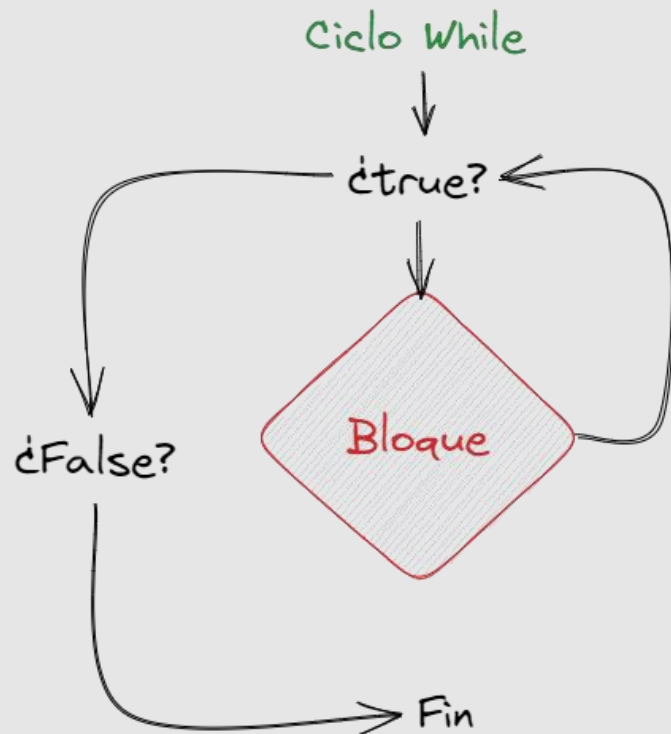
Ciclo while

Ciclo do while

Ciclo for

Ciclo while

Es una estructura de **control de flujo**, cuya función es ejecutar un bloque de código, **mientras** se **cumpla** una determinada **condición**.



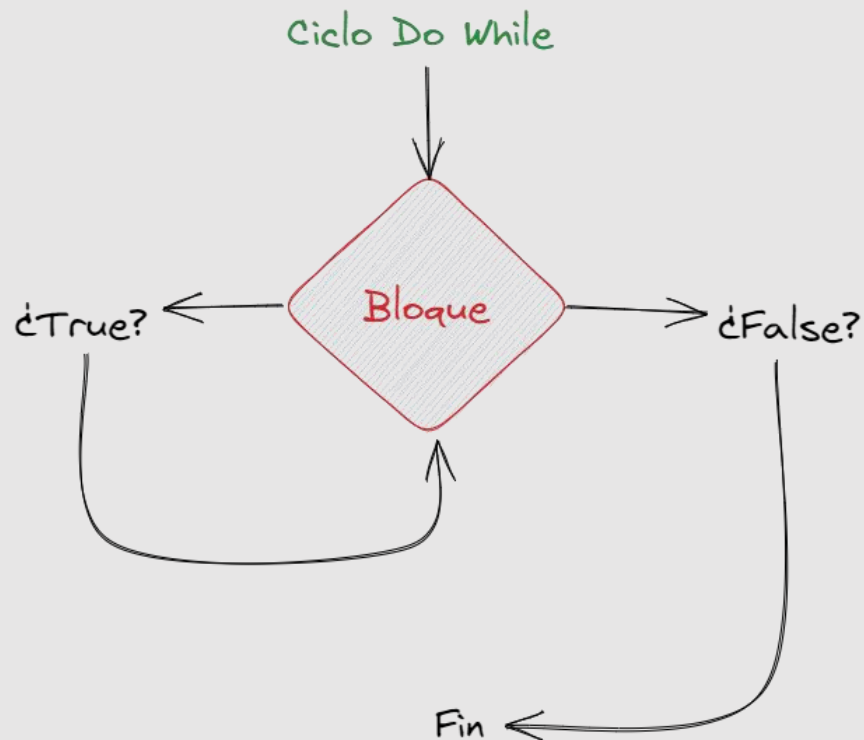
```
1 let state = true
2 let counter = 0
3
4 while (state) {
5
6     if (counter === 10) {
7         state = false
8     }
9
10    console.log(`El contador es: ${counter}`)
11
12    counter++
13 }
```

Después de ejecutar el código, la condición se evalúa de nuevo, y si sigue siendo **verdadera**, el código dentro de las llaves se ejecuta de nuevo. Este proceso se repite hasta que la condición se evalúa como **falsa**.

Ten en cuenta que, si la condición es falsa desde el principio, el código dentro de las llaves **nunca** se **ejecutará**.

Ciclo do while

Es una **estructura** de control de **flujo** que es similar al ciclo **while**, con la diferencia de que el bloque de código dentro del ciclo se **ejecuta al menos una vez** antes de verificar la condición.



```
1  do {  
2  
3      console.log('Hola Mundo')  
4  
5  } while (false)  
6
```

```
1  let state = false  
2  
3  do {  
4  
5      console.log(`El estado es ${state}`)  
6  
7  } while (state)
```

Aunque no es muy famoso ni muy utilizado, es interesante que conocer que existe. En algunos casos tendrá un uso específico y real.

Declaraciones y expresiones

En JavaScript, existen dos tipos de elementos fundamentales para escribir código: **expresiones** y **declaraciones**. Aunque estos términos pueden parecer confusos al principio, son conceptos clave para comprender cómo funciona JavaScript.

Declaraciones

Expresiones

Las declaraciones en JavaScript son sentencias que definen la creación de una variable, función o clase y producen un valor.

Las expresiones pueden ser tan **simples** como un solo número o una cadena de texto, o tan **complejas** como el cálculo de una operación matemática, la evaluación de diferentes valores o la llamada a una función.



```
1  2 + 14 // → 16
```

Podríamos decir que **las declaraciones son como las instrucciones que le damos a JavaScript para que haga algo**. Por ejemplo, una declaración de variable es una sentencia que le da un nombre y un valor a una variable.

El siguiente código es un ejemplo de una declaración de variable:



```
1 let nombreDocente = "Diego Palacio"
```

Este código no lo puedes usar con el método `console.log`, ya que no produce ningún valor. Si lo intentas, obtendrás un error.

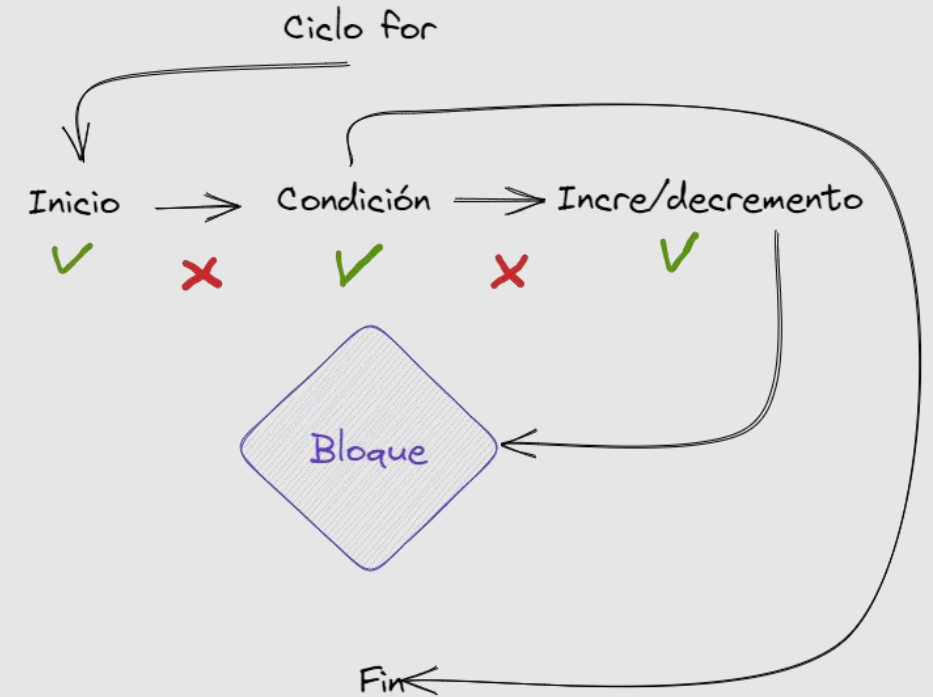
La diferencia entre declaraciones y expresiones es importante, ya que **no se puede usar una declaración donde se espera una expresión**, y viceversa.

Por ejemplo, en las estructuras de control **if** y **while**. Ambas esperan una expresión que se evalúa a un valor booleano. Por lo tanto, no se puede usar una declaración en su lugar.

Ciclo for

Es una estructura de **control** de **flujo** que se utiliza cuando se conoce la **cantidad** de veces que se debe **repetir** un bloque de código. Es el más **común** y **sencillo**.

La estructura de control **for** en JavaScript es muy útil para ejecutar una serie de **instrucciones** un número determinado de veces. A diferencia de **while**, que usa una **condición** para determinar si se ejecuta o no el bloque de código, **for** usa un contador que se **incrementa** en cada **iteración** hasta que se cumple una condición.



```
1 // Incremento
2
3 for (let i = 0; i < 10; i++) {
4
5     console.log(i);
6
7 }
```

```
1 // Decremento
2
3 for (let i = 10; i ≥ 0; i--) {
4
5     console.log(i);
6
7 }
```

Practica lo aprendido

Pon en práctica lo visto sobre ciclos resolviendo los siguientes ejercicios:



Imágenes: Freepik y Flaticon

Ejercicio 1

Determina si un número es perfecto o no.

Un número perfecto es un número entero positivo que es igual a la suma de sus divisores propios positivos.

Ejercicio 2

Ahora, a parte del número perfecto, se necesita calcular el factorial de un número.

El factorial de un entero positivo n , el factorial de n o n factorial, se define en principio como el producto de todos los números enteros positivos desde 1 (es decir, los números naturales) hasta n .

Ejercicio 3

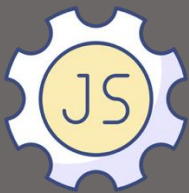
Crea las tablas de multiplicar del 1 al 10.



Ejercicio 4

Determina si una palabra o frase dada es o no un palíndromo.

Un palíndromo es una palabra o frase que se lee igual de izquierda a derecha que de derecha a izquierda.



Practica lo aprendido

Ejercicio 5

Crea un programa que pida al usuario introducir números.

El ciclo continuará pidiendo números hasta que el usuario introduzca el número 0, momento en que el programa termina y muestra un mensaje de despedida.

Ejercicio 6

Una empresa de seguridad tiene como reto desarrollar un *software* para validar contraseñas, necesita que el *software* solicite contraseñas a los usuarios hasta que esta sea correcta, debe mostrar alertas cada vez que la contraseña sea incorrecta y una alerta cuando sea correcta.

Ejercicio 7

Calcula la suma de todos los números pares hasta un número n dado.

Ejercicio 8

Calcula cuántos números impares hay entre 1 y un número n dado.



Esta guía fue elaborada para ser utilizada con fines didácticos como material de consulta de los participantes en el diplomado virtual en PROGRAMACIÓN EN JAVASCRIPT del Politécnico de Colombia, y solo podrá ser reproducida con esos fines. Por lo tanto, se agradece a los usuarios referirla en los escritos donde se utilice la información que aquí se presenta.

GUÍA DIDÁCTICA 2

M3-DV114-GU02

MÓDULO 2: STRINGS, OBJETOS, ESTRUCTURAS DE CONTROL Y CICLOS

© DERECHOS RESERVADOS - POLITÉCNICO DE COLOMBIA, 2025
Medellín, Colombia

Proceso: Gestión Académica Virtual

Realización del texto: Diego Alejandro Palacio, docente

Revisión del texto: Comité de Revisión

Diseño: Comunicaciones

Editado por el Politécnico de Colombia

